NJOY EMLAK

Real Estate Portfolio Management System

TECHNICAL REPORT

Project	Njoy Emlak — Real Estate Portfolio Management System
Database Engine	SQLite
Data Source	njoy.sahibinden.com (Live Active Portfolio)
Date	April 2025

TEAM MEMBERS

Altay Yeles	Alper Güner	Mustafa Ulaş Karaaslan	Mehmet Can Öztürk	Emre Aslan

1. Introduction and Project Idea

This project was initiated to design a relational database for the actively operating Istanbul-based real estate company Njoy Emlak (njoy.sahibinden.com), owned by team member Alper Güner's uncle. The system consolidates listing, agent, and property data into a single normalized database and enables business intelligence reporting from that data.

All listing data in the database was manually collected from njoy.sahibinden.com through data scraping. Ten active listings, along with all of their interior features, exterior features, and orientation details, were entered into the system. Because the data comes from a live source, no artificial or fabricated content was used.

2. Problem Statement and Importance

2.1 Problem Statement

Small and mid-sized real estate offices struggle to manage listing, agent, and feature data in an organized way as their portfolio grows. Dependency on scattered Excel files or ready-made third-party platforms leads to critical operational issues, including data redundancy, update anomalies, and insufficient reporting capabilities.

2.2 Why This Project Matters

Implementing this relational database system delivers the following concrete benefits:

- Centralized Data Management: All listing, agent, and feature data is stored in one place — updates are made at a single point.

- Powerful Filtering: Instant queries by budget, area, district, and property features (elevator, AC, orientation, etc.) become possible.
- Performance Analysis: The number of listings per agent and the total portfolio value can be calculated instantly.
- Scalability: New listing types, feature categories, or staff members can be added to the system with ease.
- Platform Independence: SQLite's file-based structure works in any environment and integrates with any application.

3. Database Schema Design

3.1 Core Tables (3NF Normalized)

Five core tables have been designed targeting Third Normal Form (3NF). Relationships between tables are defined through Foreign Keys. The many-to-many (M:N) relationship between Listings and Features is resolved via a separate junction table.

Table	Key Columns	Description
Ekip (Team)	DanismanID (PK, AUTO) AdSoyad, Unvan, Telefon	2 records — agent and store owner.
Emlaklar (Listings)	IlanID (PK), DanismanID (FK) Fiyat, İlce, EmlakTipi BrutM2, NetM2, OdaSayisi, Aktif	10 real listings — IlanID 1000–1009. Aktif column enables soft-delete.
Ozellik_Kategorileri	KategoriID (PK, AUTO) KategoriAdi	3 categories: Interior Feature, Exterior Feature, Orientation.
Ozellikler (Features)	OzellikID (PK, AUTO) KategoriID (FK), OzellikAdi	81 features — 53 interior, 24 exterior, 4 orientation directions.
Emlak_Ozellikleri	IlanID (FK), OzellikID (FK) [Composite PK]	Junction table — 246 listing–feature mappings resolving M:N.

3.2 Extended Tables (User & Audit System)

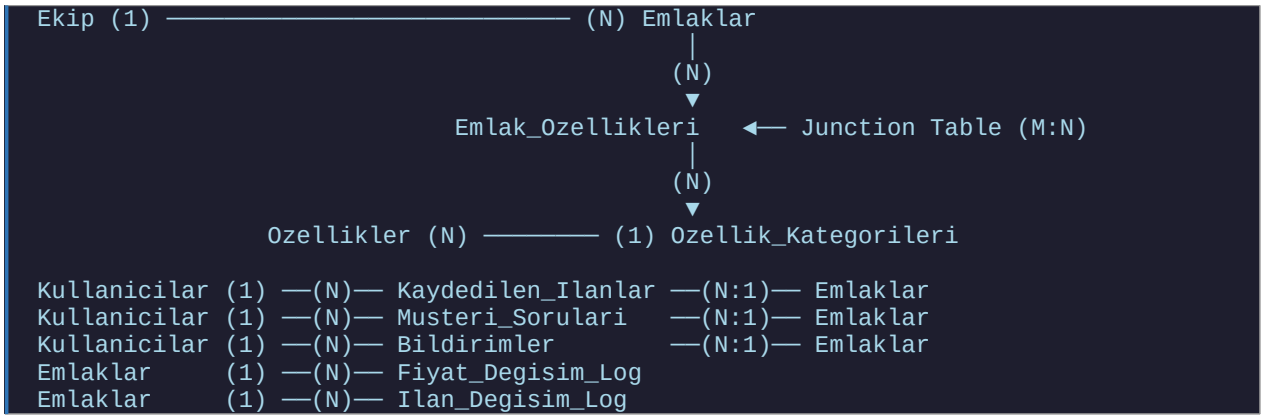
The following tables were added to support user authentication, notification delivery, customer interactions, and a full audit trail for listing changes.

Table	Key Columns	Description
Kullanicilar	KullaniciID (PK), Email (UNIQUE) SifreHash, Rol, KayitTarihi	Customer and admin accounts. Passwords stored as bcrypt hashes.
Kaydedilen_Ilanlar	KullaniciID (FK), IlanID (FK) [Composite PK], KayitTarihi	Saved/favorited listings per user.
Musteri_Sorulari	SoruID (PK), KullaniciID (FK) IlanID (FK), SoruMetni CevapMetni, Durum	Customer questions linked to listings. Admin can reply; status tracks open/answered.

Table	Key Columns	Description
Bildirimler	BildirimID (PK), KullaniciID (FK) Tip, Baslik, Mesaj, Okundu	In-app notifications (price change, saved listing removed, question answered).
Fiyat_Degisim_Log	LogID (PK), IlanID (FK) EskiFiyat, YeniFiyat DegisimYuzdesi, DegisimTarihi	Automatic price-change audit log populated by Trigger.
Ilan_Degisim_Log	LogID (PK), IlanID (FK) IslemTipi, AlanAdi EskiDeger, YeniDeger, Kullanici	Full listing change log (create / update / delete / feature changes).

3.3 Relationship Diagram

The entity-relationship structure of the schema:



The detailed ERD can be found in SCHEMA.md in the project repository.

4. Implemented SQL Features

All SQL features originally planned have been fully implemented. The table below reflects the current state of the codebase.

SQL Feature	Status	Purpose
JOIN (INNER, LEFT, 4-table)	✅ Implemented	Agent–listing matching; 4-table feature filter via junction table
WHERE + IN / BETWEEN	✅ Implemented	Multi-criteria filters by district, price range, listing type
GROUP BY + HAVING	✅ Implemented	Per-agent portfolio analysis with filtered grouping
COUNT / SUM / AVG	✅ Implemented	Listing count, total portfolio value, average price per m²
ORDER BY	✅ Implemented	Sorting by price, area, listing ID
VIEW (6 views)	✅ Implemented	v_ilan_detay, v_danisman_portfoy, v_ozellikli_ilanlar, v_bolge_fiyat_analizi,

SQL Feature	Status	Purpose
		v_ilan_fiyat_siralamasi, v_fiyat_gecmisi
CTE	✅ Implemented	BolgeOzet CTE inside v_bolge_fiyat_analizi for modular region analysis
INDEX (6 indexes)	✅ Implemented	idx_emlaklar_fiyat, idx_emlaklar_ilce_fiyat, idx_ozellikler_ad, idx_emlak_ozellikleri_ozellik + 2 more
Window Functions	✅ Implemented	RANK() OVER PARTITION BY district; ROW_NUMBER() for global ranking
Pivot (CASE WHEN)	✅ Implemented	v_ilce_oda_pivot — room type cross-tabulation by district
Trigger	✅ Implemented	trg_emlaklar_fiyat_audit — auto-logs price changes to Fiyat_Degisim_Log
Transactions	✅ Implemented	BEGIN IMMEDIATE / COMMIT / ROLLBACK on all write operations
EXPLAIN QUERY PLAN	✅ Implemented	Performance analysis endpoint with query plan output
Soft Delete	✅ Implemented	Emlaklar.Aktif column — logical deletion preserving audit history

5. Sample SQL Queries

Query 1 — Full Listing Report (INNER JOIN)

Lists all active listings with the responsible agent's name and contact information. This is the primary output query for clients or office managers.

```
SELECT E.Baslik, E.Fiyat, E.İlce, E.EmlakTipi,
       K.AdSoyad AS Danisman_Adi, K.Telefon
FROM   Emlaklar E
INNER JOIN Ekip K ON E.DanismanID = K.DanismanID
WHERE  E.Aktif = 1;
```

Query 2 — Budget and Region Filtering (WHERE, IN)

Retrieves apartments with a budget of 40,000 TL or less in the Şişli and Beyoğlu districts.

```
SELECT Baslik, Fiyat, İlce, Mahalle
FROM   Emlaklar
WHERE  Fiyat <= 40000
  AND  İlce IN ('Şişli', 'Beyoğlu')
  AND  Aktif = 1
ORDER BY Fiyat DESC;
```

Query 3 — Agent Portfolio Analysis (GROUP BY + SUM + COUNT)

A performance report designed for management. The LEFT JOIN ensures agents with no listings are also included. Each agent's listing count and total portfolio value (TRY) is calculated automatically.

```
SELECT K.AdSoyad,
       COUNT(E.IlanID)  AS ToplamIlanSayisi,
       SUM(E.Fiyat)     AS ToplamPortfoyDegeri
FROM   Ekip K
LEFT JOIN Emlaklar E
```

```
       ON K.DanismanID = E.DanismanID AND E.Aktif = 1
GROUP BY K.AdSoyad;
```

Query 4 — Feature-Based Filtering (4-Table JOIN + DISTINCT)

Lists properties with heat insulation or air conditioning along with their feature category. This query chains four tables through JOIN operations, demonstrating the practical value of the junction table design.

```
SELECT DISTINCT E.Baslik, E.Fiyat,
                Oz.OzellikAdi, Kat.KategoriAdi
FROM   Emlaklar E
INNER JOIN Emlak_Ozellikleri EO      ON E.IlanID      = EO.IlanID
INNER JOIN Ozellikler Oz             ON EO.OzellikID  = Oz.OzellikID
INNER JOIN Ozellik_Kategorileri Kat  ON Oz.KategoriID = Kat.KategoriID
WHERE  Oz.OzellikAdi IN ('Isı Yalıtımı', 'Klima')
  AND  E.Aktif = 1;
```

Query 5 — Rent Cost per m² (Arithmetic)

Calculates the monthly rent cost per gross and net square meter for each listing, enabling quick price-per-m² comparison across the portfolio.

```
SELECT Baslik, Fiyat, BrutM2, NetM2,
       ROUND(Fiyat / NULLIF(BrutM2, 0), 0) AS BrutM2_Fiyati,
       ROUND(Fiyat / NULLIF(NetM2, 0),  0) AS NetM2_Fiyati
FROM   Emlaklar
WHERE  BrutM2 > 0 AND NetM2 > 0 AND Aktif = 1
ORDER BY NetM2_Fiyati DESC;
```

Query 6 — Region Price Analysis (CTE)

Uses a Common Table Expression to compute per-district statistics in a modular, readable form. This query is exposed as the v_bolge_fiyat_analizi VIEW.

```
WITH BolgeOzet AS (
  SELECT İlce,
         COUNT(*)                              AS IlanSayisi,
         ROUND(AVG(Fiyat), 2)                 AS OrtalamaFiyat,
         MIN(Fiyat)                            AS EnDusukFiyat,
         MAX(Fiyat)                            AS EnYuksekFiyat,
         ROUND(AVG(Fiyat / NULLIF(NetM2,0)),2) AS OrtalamaNetM2Fiyati
  FROM   Emlaklar
  WHERE  Aktif = 1
  GROUP BY İlce
)
SELECT *, ROUND(EnYuksekFiyat - EnDusukFiyat, 2) AS FiyatAraligi
FROM   BolgeOzet;
```

Query 7 — Listing Price Ranking (Window Functions)

Ranks each listing both within its district and globally using RANK() and ROW_NUMBER() window functions.

```
SELECT IlanID, Baslik, Fiyat, İlce,
       RANK()      OVER (PARTITION BY İlce ORDER BY Fiyat DESC) AS IlceIciFiyatSirasi,
       ROW_NUMBER() OVER (ORDER BY Fiyat DESC)                  AS GenelFiyatSirasi
FROM   Emlaklar
WHERE  Aktif = 1;
```

6. Web Application & REST API

A full-featured Flask web application (webapp.py) with an Art Deco–inspired dark UI has been developed on top of the NjoyRepository class defined in app.py. The application is accessible at http://localhost:5000.

Module	Description
İlanlar (Listings)	Browse all active listings as cards — sortable by price, m², or listing ID.
Arama (Search)	Advanced search with max-price, district, feature, and limit filters.
Danışmanlar (Agents)	Agent portfolio stats with animated progress bars.
Analiz (Analytics)	CTE-powered region analysis, window function rankings, and room-type pivot.
İlan Geçmişi (History)	Full audit trail from Ilan_Degisim_Log and Fiyat_Degisim_Log via VIEWs.
Performans (Benchmark)	One-click query benchmark timing + EXPLAIN QUERY PLAN output.
Admin Paneli	Full CRUD for listings (create, update, price change, soft-delete). Access via admin@njoyemlak.com.
Kullanıcı Girişi / Kayıt	Customer registration, login (bcrypt hashed passwords), session management.
Kaydedilen İlanlar	Logged-in users can save/unsave listings; changes trigger in-app notifications.
Soru / Cevap	Customers submit questions on listings; admin answers through the panel.
Bildirimler	In-app notification system — price changes, removed listings, answered questions.

6.1 REST API Endpoints

The web application exposes a complete JSON REST API:

Endpoint	Description
GET /api/listings	All active listings with optional sort_by and limit
GET /api/search	Filtered search — max_price, district, feature, limit
GET /api/stats	Agent portfolio statistics (COUNT, SUM, AVG)
GET /api/analytics	CTE, window function, and pivot analysis results
GET /api/benchmark	Query performance timing across all major queries
GET /api/explain	EXPLAIN QUERY PLAN output for the search query
GET /api/price-history	Price change log from v_fiyat_gecmisi VIEW
GET /api/listing-history	Full listing audit log from v_ilan_gecmisi VIEW
GET /api/meta	Available districts, features, and sort options
POST /api/admin/listings	Create new listing (admin only)
PUT /api/admin/listings/:id	Update listing fields (admin only)
DELETE /api/admin/listings/:id	Soft-delete listing; notifies saved-listing watchers
POST /api/admin/listings/:id/price	Update price — triggers audit log automatically
POST	Admin answers a customer question

Endpoint	Description
/api/admin/questions/:id/answer	
POST /api/account/save-listing	Save a listing (customer only)
DELETE /api/account/save-listing/:id	Unsave a listing
POST /api/account/questions	Submit a question on a listing
POST /api/account/notifications/read	Mark all notifications as read

7. Python CLI (app.py)

A comprehensive command-line interface (app.py, 1,752 lines) provides full database access without the web server. All operations go through the NjoyRepository class which reuses the same SQLite connection logic.

```
# List listings
python3 app.py list --limit 10 --sort-by fiyat_desc --output table

# Filtered search
python3 app.py search --max-price 40000 --district Beyoğlu --feature 'Isı Yalıtımı'

# Agent portfolio stats
python3 app.py stats

# Advanced analytics (CTE, Window, Pivot)
python3 app.py analytics

# Price update — triggers Fiyat_Degisim_Log automatically
python3 app.py update-price 1000 41000

# Query performance benchmark
python3 app.py benchmark --output table

# SQLite backup
python3 app.py backup
```

8. Quality, Testing, and CI/CD

The project includes a full continuous integration pipeline via GitHub Actions (.github/workflows/ci.yml) with unit tests and linting.

- CI Workflow: Runs automatically on every push and pull request to the main branch.
- Unit Tests: Located in the tests/ directory, executed with make test.
- Linting: Code quality enforced via make ci (lint + test combined).
- Contribution Guide: CONTRIBUTING.md documents branch naming, commit message conventions, and PR requirements.
- Makefile: make ci runs the full pipeline; make test runs unit tests only.

```
# Run linting + tests
make ci

# Run unit tests only
make test
```

9. Conclusion

In this project, the normalized relational database schema — built with real-world data from the active Njoy Emlak portfolio on njoy.sahibinden.com — has been successfully designed, implemented, and extended into a full-stack application.

All originally planned SQL features (JOIN, WHERE, GROUP BY, CTE, VIEW, INDEX) have been implemented, and the scope was further extended to include Window Functions, Pivot queries, Triggers, Transactions, Soft Delete, a user authentication system, an admin panel, customer messaging, and a notification system.

GitHub Repository	Live Data Source
github.com/alpergnr/SQL-PROJECT	njoy.sahibinden.com